Lane Detection Algorithm Analysis — IPG CarMaker LKA System

1. Introduction

This document traces the evolution of the lane-detection pipeline developed for the Lane Keeping Assist (LKA) system, running in IPG CarMaker 15.0 with MovieNX rendering. The camera is a Camera RSI sensor delivering 640×480 RGB frames over TCP (port 2210) at ~30 Hz where the vehicle target speed is 30 km/h.

Each algorithm is evaluated on its merits (why it was selected), the problems encountered, and how those problems led to the next approach — culminating in the Bird's-Eye-View Sliding Window pipeline. While a deep learning approach can help a lot with the accuracy of the system, speaking about the time for development and performance of the processor, the Bird's-Eye-View Sliding Window pipeline was opted.

2. IPG CarMaker Environment Issues

Before analysing algorithm-specific problems, several **simulator-level issues** fundamentally limited all classical vision approaches:

2.1 Camera Exposure Settings

Parameter	Original Value	Corrected Value
ISO	100	800
F-Stop	11.0	2.8
Shutter Speed	250 Hz	250 Hz
Tone Mapping	None	Filmic

These values should be tuned according to the rendering performance of the Camera and the scene saturation levels, leaving sufficient contrast between road surface and grass in the IPG CarMaker environment.

Problem: The combination of ISO 100 + F-Stop 11.0 resulted in an extremely underexposed image (~5 stops too dark). Raw pixel values ranged from 10–30 (out of 255), making lane markings unrecognised from the road surface.

Impact: The signal-to-noise ratio between lane paint and asphalt was very less.

2.2 Tone Mapping Disabled (the process of compressing HDR to LDR)

Problem: With `ToneMapping.Mode = None`, the HDR values were less with the camera exposure settings first taken, and without the toen mapping to compress the dynamic range of teh camera, everything clipped to near-black.

Impact: Even with increased ISO/aperture, without tone mapping the full dynamic range of the scene was not being utilized in the output image.

2.3 Vehicle File Caching

Problem: CarMaker uses `.tmp` files when the vehicle is open in the editor. Changes made to the main vehicle file are not reflected unless all `.tmp` files are also updated, or CarMaker is fully restarted. The GUI exposure changes were not saved to disk.
It is safe to delete them because they're just temporary working copies. The actual data lives in the main file
(Data/Vehicle/Examples/Demo_IPG_CompanyCar_SensorGroundTruth_LanesAndRoots).

2.4 Sun/Environment Configuration

The TestRun specified `Env.Sun.Elevation = 45.0` (midday sun) and `Env.StartTime.Hour = 12`, yet the rendered scene appeared as night. This disconnect between logical environment settings and actual rendered brightness pointed to the exposure/tonemapping chain being the bottleneck, not the scene lighting itself.

3. Algorithm Analysis

3.1 CLAHE + Percentile Threshold

Why it was chosen:
- CLAHE (Contrast Limited Adaptive Histogram Equalisation) is designed for low-contrast images
- Locally adaptive — enhances contrast even in dark regions
- Percentile-based thresholding adapts to the actual brightness distribution
- Combined approach: enhance → then threshold the enhanced image

Problems encountered:
1. **Noise amplification**: CLAHE's tile-based histogram equalisation amplified sensor noise and compression artefacts as aggressively as the lane signal.
2. **Percentile instability**: The "top N%" of pixels shifted between road texture, grass, and lane markings depending on the frame composition.

Conclusion: CLAHE is a preprocessing tool, not a solution. It cannot create contrast that doesn't exist in the raw data. With pixel values 10–30, there is simply insufficient information.

3.2 Hough Transform

Why it was chosen:
- Classic line detection algorithm with well-understood parameters
- Direct output of line segments with angle and position
- Well-suited for detecting straight lane markings
- Extensive OpenCV documentation and examples
- Simple pipeline: grayscale → Canny → Hough Lines → filter by angle

Problems encountered:
1. **Dashed lines fragmented**: HoughLinesP detected each dash as a separate short segment. Connecting them required complex grouping logic with distance and angle thresholds.
2. **Curves broke the model**: On curved road sections, lane markings are not straight lines. Hough fundamentally assumes linearity.
3. **Angle thresholding unreliable**: Filtering lines by angle (e.g., 20°–50°) discards valid lane lines on curves while accepting road-edge artefacts at similar angles.
4. **No positional error output:** Hough gives line equations, not lateral offset. Converting to a lane-centre offset required additional geometry that was fragile.
5. **Sensitivity to edge quality:** Relied entirely on Canny edge map quality, which was poor due to the dark exposure.

Conclusion: Structural limitations (dashed lines, curves) make Hough unsuitable regardless of image quality.

3.3 Adaptive Thresholding

Why it was chosen:
- Computes threshold per-pixel based on local neighbourhood mean
- Handles uneven illumination (shadows, gradients across the road)
- Parameter C allows biasing toward bright-only features (lane markings)
- BlockSize controls the locality scale — can be tuned to lane width

Problems encountered:
1. **Near-black image saturation:** When the threshold droped below zero (because the image is so dark, which makes the mean go negative), every single pixel passes, and the binary mask is 100% white, which lead to teh algorithm finding noise everywhere, which inturn caused no lane detection.
2. **Reduced C still noisy**: The threshold values were too low or too high, thus finding the C value which could cleanly separate the lane paint at 25 pixels from teh road at 15 to 20 pixels using a local mean of 18 pixels was practically impossible.
3. **Uniform dark regions:** In areas where all pixels are ~15-20 (road surface), the local mean equals the pixel values, so ANY negative C makes everything pass.

Conclusion: Adaptive thresholding assumed locally varying illumination with consistent signal contrast. In a uniformly dark image, the "adaptation" provides no benefit and produces low-quality results.

3.4 Ridge Detection

Why was it considered
- Ridge detection algorithms (Frangi was selected) are specifically designed to detect tubular/curvilinear structures — exactly what lane lines are
- Based on eigenvalue analysis of the Hessian matrix — detects the "ridge-like" intensity profile of a bright line on a dark background
- Orientation-independent: responds to ridges at any angle without prior knowledge of lane direction
- Stronger theoretical foundation for line detection than simple thresholding

Problems:

1. **Contrast dependency:** The Hessian eigenvalues are proportional to the second derivative of intensity. With pixel values 10–30, the second derivatives would have been too low for the ridge response to be at or below noise floor.
2. **Road-grass boundary response** Since the lane lines and the road boundary overlap in width, no sigma values could selectively detect lanes while rejecting the road edge.
3. **Dark image degradation** With boosted images (5× multiply), quantisation artefacts create artificial ridge-like patterns in flat regions. The Hessian responds to these as valid ridges.
4. **Dashed line gaps**: Ridge detection finds ridges per-pixel — it doesn't connect dashed segments. Still requires a post-processing stage (e.g., sliding window or connected components) to build continuous lane lines.

Conclusion Ridge detection require sufficient signal contrast. In the CarMaker dark-frame scenario (pixel values 10–30), the Hessian eigenvalues was too weak to reliably separate lane ridges from noise ridges. Additionally, computational cost makes it less suitable for real-time 30 Hz operation compared to simpler methods. It remains a strong candidate IF the exposure problem was solved.

3.5 Bird's-Eye-View + Sliding Window (Final Classical Approach)

Why it was chosen:
- **Perspective removal**: BEV transform eliminates perspective foreshortening, making lane lines appear as near-vertical strips regardless of distance
- **Histogram initialisation:** Bottom-half histogram provides robust starting positions for lane search

- **Sliding window:** Tracks lanes upward window-by-window, naturally connecting dashed lines
- **Position-based output:** Directly computes lateral error in metric space (BEV calibrated to meters)
- **No angle assumptions:** Works on straight roads and curves equally
- **Fit caching**: Stored polynomial fits bridge detection gaps (up to 15 frames)
- **Industry standard:** Used in production ADAS systems and academic literature

Problems encountered:
1. **BEV calibration sensitivity:** The source trapezoid and pixels-per-meter ratio must be calibrated from a known-good frame. Without adequate brightness, calibration was impossible.
2. **Histogram failure on empty masks**:When the binary mask is all-zero or all-white, the histogram peak detection returns index 0 or a noise peak — producing garbage starting positions (this can be solved by taking previous lane positions as starting points).
3. **Single-lane fallback fragility**: When only one lane is detected, the center is inferred by adding half the lane width — but this assumes the lane width is exactly 3.5m, which may not hold on all road segments.

Verdict: The BEV + Sliding Window architecture is sound and well-suited for the task. It correctly handles dashed lines and curves, and provides metric error output.

5. Observation: PID vs Pure Pursuit + PID for Lateral Control

A standalone PID controller reacts to lateral error after it occurs, with no knowledge of vehicle geometry, speed, or road curvature. With a maximum steering limit of ±0.524 rad (make sure that all the quantities are in degrees or rad and not mixed up) and a rate limit of 0.03 rad/frame, the controller had no mechanism to anticipate curves — only correct for already-accumulated error. This made it prone to oscillation, integral windup during vision dropouts, and poor curve tracking.

Pure Pursuit addresses these limitations geometrically — it computes the steering angle required to follow an arc from the vehicle's current position to a lookahead point 5.0m ahead on the detected lane, using the vehicle wheelbase of 2.622m and a steering ratio of 16:1. This provides a stable, physically meaningful baseline command. At the target speed of 30 km/h, Pure Pursuit naturally scales its response with vehicle dynamics — something fixed PID gains cannot achieve. The combination is more robust: Pure Pursuit handles geometry and prediction, PID handles correction.

6. How Deep Learning Solves the Problem

6.1 Why Classical Methods Failed

All classical methods share a fundamental limitation: they operate on **pixel intensity values** and require a minimum contrast ratio between lane markings and road surface. When the camera delivers pixel values of 10–30 with a lane-vs-road difference of ~10 intensity units, no amount of histogram stretching, adaptive thresholding, morphological filtering, or ridge detection can reliably separate the two.

6.2 Deep Learning Advantages

A trained neural network (e.g., LaneNet, SCNN, UFLD, PINet) overcomes these limitations through:

1. **Learned feature representations:** The network learns to detect lanes from contextual patterns — road texture, perspective convergence, colour gradients — not just absolute brightness. It can recognise a lane marking that is only 10 intensity units brighter than the road because it has seen plenty of similar examples during training.

2. **Robustness to exposure variation:** Data augmentation during training (random brightness, contrast, gamma) teaches the network to detect lanes across a wide range of imaging conditions — including underexposed and overexposed frames.

3. **Semantic understanding**: The network learns that lanes follow certain geometric patterns (parallel, converging in perspective, continuous along the road direction) and can infer lane positions even when visual evidence is sparse.

6.3 Suitable Deep Learning Architectures (ASSUMPTIONS)

Model	Architecture	Advantages for This Use Case
SCNN (Spatial CNN)	Message passing between adjacent pixel rows/columns	Excels at detecting long, thin structures (lanes) even with occlusion
UFLD (Ultra-Fast Lane Detection) V2	Row-wise classification	Extremely fast (~300 FPS), suitable for 30 Hz real-time requirement
LaneATT	Anchor-based attention	Good balance of speed and accuracy, handles curves well
PINet	Point instance network	Detects lane points directly, works with dashed lines

CLRNet	Cross-layer refinement	State-of-art accuracy, multi-scale feature fusion

6.4 Implementation Approach

Camera Frame (640×480 RGB, even dark)

↓

Pre-trained Lane Detection Model (e.g., UFLD or SCNN)

↓ - Fine-tuned on the CarMaker rendered frame

Lane Polynomial Coefficients (or point lists)

↓

Lateral Error Computation (same as BEV approach)

↓

Pure Pursuit + PID Controller

7. File-by-File Breakdown

7.1. LKA_Main.py

Purpose: It runs the main control loop, acting as the bridge between the perception module and the CarMaker simulation.

What it does:

- Connects to CarMaker: Uses the IPG CarMaker Python API (APO interface) to take over control of the vehicle's steering, gas, and brake.
- Fetches Data: Grabs camera frames via TCP and reads vehicle speed and yaw rate directly from the simulation.
- Control Logic (Pure Pursuit + PID: It uses Pure Pursuit to calculate a baseline steering angle and calculates the lookahead distance to a point on the road.
- It adds a PID Controller on top of this to reduce the steady-state errors and ensure smooth lane centring.
- Actuation: Rate-limits the steering command to prevent jerky movements and sends the final road-wheel angle back to CarMaker.akdown

7.2. sensor_fusion.py

Purpose: This is the computer vision module. It takes raw images from the virtual camera and mathematically figures out where the lane lines are and how far off-centre the car is.

What it does:

- Bird's-Eye View (BEV): Warps the perspective of the camera so it looks like a top-down view. This removes perspective distortion and makes the lanes parallel.
- Sliding Window Search: Scans the image from the bottom up to track the pixels belonging to the left and right lane lines.
- Polynomial Fitting: Fits linear mathematical equations to the lane pixels to calculate the exact centre of the lane and the lateral deviation (error) in meters.

7.3. config.py

Purpose: This file centralises all the tunable parameters in one place.

What it does:

- Keeps Code Clean: Instead of hardcoding numbers in the logic files, everything is pulled from here.
- Tuning Hub: To change the vehicle speed, adjust the PID aggressiveness, or modify the camera resolution.
- Mapping: Contains the exact string names for CarMaker's Data Variable Access (DVA), ensuring the script reads and writes to the correct virtual sensors.

7.4. Lka_utils.py

Purpose: A helper module that keeps the main control loop clean by handling background tasks.

What it does:

- RMSE Tracker: Calculates the Root Mean Square Error (RMSE) to score how well the car is staying in the centre of the lane.
- DNF Watchdog: Monitors if the car leaves the lane for too long.
- Data Logger: Saves all the telemetry (steering angle, error, speed) to a CSV file every tick, allowing to graph and analyse the run later.